Exploring Computer Science Concepts

Via ACSL Competitions

Number Systems

- Decimal
  - base 10
- Binary
  - base 2
- Octal
  - base 8
- HexaDecimal
  - base 16

Digits per base

- Decimal
  - Ten digits
  - 0 1 2 3 4 5 6 7 8 9
- Binary
  - Two Digits
  - 0 1
- Octal
  - 8 digits
  - 0 1 2 3 4 5 6 7
- Hexadecimal
  - 16 digits
  - 0 1 2 3 4 5 6 7 8 9 A B C D E F

Counting in Decimal

- Increment the lowest place value (right most digit)
- When last digit is reached
  - Set the current column to 0
  - Increment the column on the left by 1
- Lets count with decimal
  - 0 to 10
  - 95 to105
- How frequently do we add a new digit ?

Decimal	Decimal
0	95
1	96
2	97
3	98
4	99
5	100
6	101
7	102
8	103
9	104
10	105

Counting in other bases

- Counting in binary
  - We add a new digit frequently
    - At 2 , 4, 8, 16 … decimal values
- Counting in Octal
  - We add a new digit at every
    - At 8, 64,128 …
- Counting in HexaDecimal
  - We add a new digit at every
    - At 16, 256 … values

Decimal	Binary	Octal	HexaDecimal
0	0	0	1
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Place Values

- Decimal
  - *We deal in power of 10s*
  - 2452 = 2 * 1000 + 4 * 100 + 5 * 10 + 2 * 1
  - 101 = 1 * 100 + 0 * 10 + 1 * 1
- Binary
  - *We deal in power of 2s*
  - 111 = 1 * 4 + 1 * 2 + 1 * 1
  - 1010 = 1 * 8 + 0 * 4 + 1 * 2 + 0 * 1

Place values
Converting binary → decimal

Decimal	Binary
2452 → 2 x 10^3 = 2000 4 x 10^2 = 400 5 x 10^1 = 50 2 x 10^0 = 2 = 2452	111 → 1 x 2^2 = 1 x 4 = 4 1 x 2^1 = 1 x 2 = 2 1 x 2^0 = 1 x 1 = 1 = 7
101 → 1 x 10^2 = 100 0 x 10^1 = 0 1 x 10^0 = 1 = 101	1010 → 1 x 2^ 3 = 1 x 8 = 8 0 x 2^ 2 = 0 x 4 = 0 1 x 2^ 1 = 0 x 2 = 2 0 x 2^ 0 = 0 x 1 = 0 = 10
756 →	1000 →

Converting Binary to Decimal

128	64	32	16	8	4	2	1	
	1	0	0	1	0	0	1	= 64 + 16 + 1 = 81
1	0	0	0	0	1	1	1	= 128 + 4 + 2 + 1 = 135
1	1	1	0	0	0	0	0	= ?

Converting Decimal to Binary

128	64	32	16	8	4	2	1	
								= 22
								= 33
								= 130

Exercises

- Convert following Decimal numbers to binary
  - 127_{10}
  - 128_{10}
  - 129_{10}
  - 255_{10}
  - 256_{10}
- Convert following Binary numbers to Decimal
  - 101101_2
  - 1110_2
  - 1111_2
  - 0110_2

Challenge

- How does binary addition and subtraction work ?

Place values
Converting octal,hexadecimal→decimal

Octal → Decimal	HexaDecimal → Decimal
777 → 7 x 8^2 = 7 x 64 = 448 7 x 8^1 = 7 x 8 = 56 7 x 8^0 = 7 x 1 = 7 = 511	2AB → 2 x 16^2 = 2 x 256 = 512 A x 16^1 = 10 x 16 = 160 B x 16^0 = 11 x 1 = 11 = 683
137 → 1 x 8^2 = 1 x 64 = 64 3 x 8^1 = 3 x 8 = 24 7 x 8^0 = 7 x 1 = 7 = 95	101 → 1 x 16^2 = 1 x 256 = 256 0 x 16^1 = 0 x 16 = 0 1 x 16^0 = 1 x 1 = 1 = 257
756 →	4A3 →

Converting Decimal to Octal

$(312)_{10} \rightarrow (?)_8$

312 / 8 → quotient : 39 , Remainder 0

39 / 8 → quotient : 4 , Remainder 7

4 / 8 → quotient : 0 , Remainder 4

$\mathbf{(470)_8}$

$(112)_{10} \rightarrow (?)_8$

112 / 8 → quotient : 14 , Remainder 0

14 / 8 → quotient : 1 , Remainder 6

1 / 8 → quotient : 0 , Remainder 1

$\mathbf{(160)_8}$

Exercises

- Convert following Decimal numbers to Octal
  - 111_{10}
  - 88_{10}
  - 511_{10}
  - 512_{10}
  - 513_{10}
- Convert following Octal numbers to Decimal
  - 45_8
  - 77_8
  - 100_8
  - 101_8

Converting Decimal to Hexadecimal

$(312)_{10} \rightarrow (?)_8$

312 / 16 → quotient : 19 , Remainder 8

19 / 16 → quotient : 1 , Remainder 3

1 / 16 → quotient : 0 , Remainder 1

$\mathbf{(138)_{16}}$

$(112)_{10} \rightarrow (?)_8$

112 / 16 → quotient : 7 , Remainder 0

7 / 16 → quotient : 0 , Remainder 7

$\mathbf{(70)_{16}}$

Hexadecimal → Octal

$(AC)_{16}$ → $(\mathbf{1010})_2$ x 16 + $(\mathbf{1100})_2$ x 1 //Replace hexa with binary

→ $(\mathbf{1010\ 1100})_2$ // Convert to binary

→ $(\mathbf{010\ 101\ 100})_2$ // group by 3s , added extra 0s in the front

→ ($\mathbf{254})_8$// Replace each group by octal value

$(1EF)_{16}$ → $(\mathbf{0001})_2$ x 256 + $(\mathbf{1110})_2$ x 16 + (1111) x 1

→ $(0001\ 1110\ 1111)_2$ // Convert to binary

→ $(\cancel{000}\ 111\ 101\ 111)_2$ // group by 3 , removed 0s in the front

→ ($\mathbf{757})_8$// Replace each group by octal value

Octal → Hexadecimal

- $(\mathbf{757})_8 \rightarrow (?)_{16}$
- $(\mathbf{254})_8 \rightarrow (?)_{16}$

Exercises

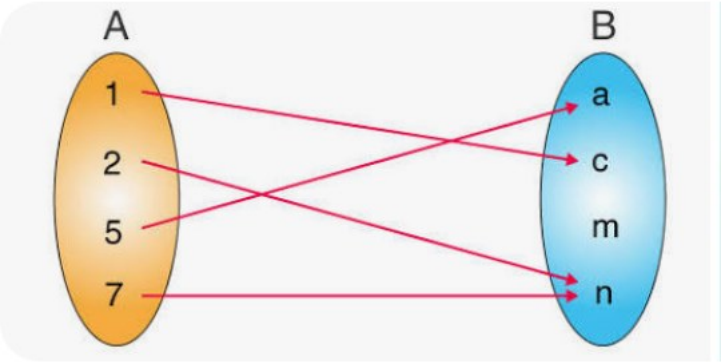
- Convert following Decimal numbers to HexaDecimal/Octal
  - 255_{10}
  - 256_{10}
  - 257_{10}
- Convert following HexaDecimal numbers to Decimal, Octal
  - 99_{16}
  - 100_{16}
  - 101_{16}

Recursion : programming

- Create factorial
  - 5! = 5 x 4 x 3 x 2 x 1 = **120**
  - 2! = **2**
- Provide sum of Fibonacci numbers using recursion
  - fibonacci(13)
    - 0 + 1 + 1 + 2 + 3 + 5 + 8 + 13 = **33**
  - fibonacci(3)
    - 0 + 1 + 1 + 2 + 3 = **7**

Relations

- A **relation** : connection/mapping between elements of two or more sets, Some characteristics …
  - It is a **mapping** as *shown in figure*
  - Can be written as **Ordered pairs**
    - {(1,c), (2,n),(5,a),(7,n)}
  - Not always unique
    - E.g. $y^2 = 4$ has multiple solutions (how many?)



- Examples
  - Numerical relationship : 4+3 = 7
  - Equation : y = 2x+3
  - Geometry : two congruent triangles
  - Set theory : A is a subset of B

Functions , mathematical kind

- A **relation** that gives exactly **one unique output for each input**

x → Rule → y :this is a function as you always get one answer

x → Rule → a, b, c :Not a function as you get multiple answers

- Representation
  - f(x) = 2x+1 : this is a function
  - g(x) = ± 3x : this is **not** a function , why?
- Follow up reading (optional): pg 11-13 : functions

Evaluating functions

- Solving
  - Substitute variables with numerals
  - Evaluate
    - Follow PEMDAS/BODMAS
- Solve for x = 0,1,2,3
  - $f(x) = 3x + 1$
  - $g(x) = 2x^2 + 3$
  - $h(x) = x^2 + 2x + 1$

Recursive Functions

- Functions calling themselves

Fibonacci numbers	$fib: \mathbb{N} \to \mathbb{N}$ $fib(n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ fib(n-1) + fib(n-2), & \text{if } n \geq 2. \end{cases}$
Factorial	**fact(n) = n * fact(n-1)** {given, *fact(1) = fact(0) =1}*
Lucas numbers (*same rule as Fibonacci but with different starting values*)	**l (n) = l(n-1) + l(n-2)** {given , *l(0) = 2, l(1) = 1}*

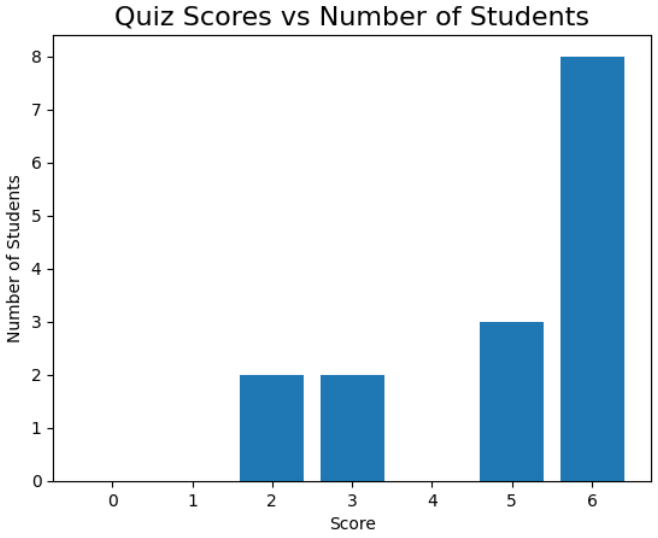
- Evaluate for ‘n’ = 2 , 4, 5, 6
- Follow up reading (optional): nested functions visualized

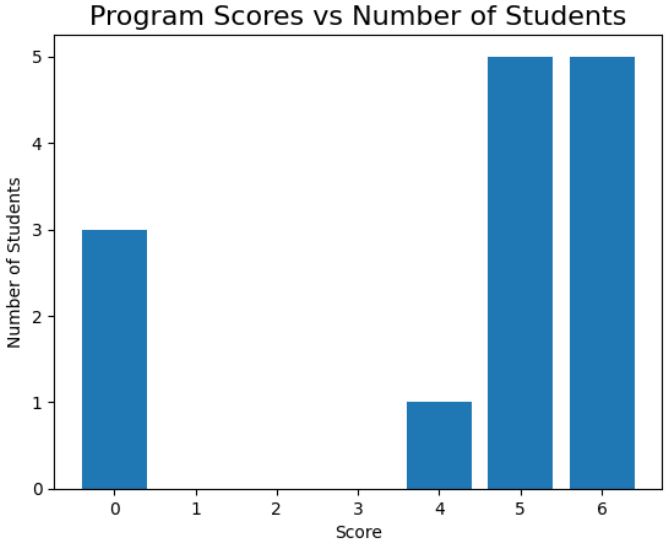
Programming, , Test #1

- Need to know
  - Data types
  - Conditionals : If/else
  - Loops : for
  - Arrays : 2 dimensional
- Practice
  - Arrays- DS (Hackerrank)
    - Do the 4 problems marked “easy”
    - Optional :“hard”, “medium” one are for extra challenge

Test 1, observations

- Great participation, nearly everyone attempted!
- Overall students did better in quiz
  - 8x full marks in quiz vs 5x full marks in programming
- 4 students got a zero in programming (with 1 not attempted)





Test 2 , analysis

- The quizzes
  - seem to be easier for students to master
  - Success cause everyone attempted, next we figure out how to raise scores there
    - Suggestions ?
- Programming
  - It was amazing attempt
  - The codes generally worked
  - But 4/15 students had zeroes.
  - Suggestions ?
- Goal :We need raise scores for everyone and move some in attempted stage
  - Note
    - Compete with yourself , Do better than your last attempt
    - Ask for help
      - on what you don’t know, to learn it.
      - And even on what you know , i.e. to master it

Topics for Test 2

- Mathematical expressions
  - Representations (infix, prefix, postfix)
  - Evaluation or expression
- Discrete Mathematics
  - Logical Operators (AND , OR , NOT, SHIFT,...)
  - Boolean logic
- Programming
  - char, string ,arrays
  - Loops
  - Conditionals
    - If/else
    - switch/case

Infix expressions

- Encounter them in grade maths. e.g.
  - (11+14)/(9 - 3) + 2
  - 3+7 / (4 * 5 - 6)
  - - 2 + 8 / 2
- Can also be written using variables
  - (a + b) / (c – d) + e
  - x + y / (g * h – k)
- Evaluated using
  - PEMDAS / BODMAS

Expression : anatomy

- Expression is made of
  - Operators (+ , - , * , /)
  - Operands (numbers, variables)
- Unary operators : - a
  - <operator> <operand>
- Binary operators (infix) : a + b
  - < operand> <operator> <operand>

Prefix/Postfix expressions

- Benefits
  - Remove ambiguity(i.e. can forget PEMDAS, BODMAS)
  - Computer friendly
  - Works with both unary/binary
  - Faster evaluation
- Expression styles
  - Infix : operator in **middle** of operands
    - (5+6)*3
    - < ộřêsắŋđ> <*operator*> <ộřêsắŋđ>
  - Prefix : operator **before** operands
    - * + 5 6 3
    - <*operator*> <ộřêsắŋđ> < ộřêsắŋđ>
  - Postfix : operator **after** operands
    - 5 6 + 3 *
    - <ộřêsắŋđ> < ộřêsắŋđ> <*operator*>

Evaluating expression,
*but first **Stacks** and **Queues***

- What is a **Stack** ?
  - This is a sequential structure of **items**
  - That are **pushed at one end**
  - **Pulled** via the **same end**
  - **LIFO** : Last in First Out
  - Examples
    - Stack of plates
    - Your turned in paper assignments
- What is a **Queue**?
  - Another sequential structure of **items**
  - that are **pushed at one end**
  - **Pulled** via the **opposite end**
  - **FIFO** : First In First Out
  - Examples
    - Queue for buying tickets
    - Traffic in one way single lane

Evaluating **prefix** expression

- Infix : (5+6)*3
  - * + 5 6 3
  - - + 2 * 3 4 / 16 ^ 2 3
- Algorithm (harder way)
  - Scan from right to left
  - Anytime you find an operator
    - Evaluate the operation using the previous 2 operands
    - Replace the <*operator*> <ộřêsắŋđ> < ộřêsắŋđ> with sêṣụľţ in the expression
  - Repeat the steps till only 1 operand is left

Evaluating postfix expression

- Evaluate
  - 2 3 1 * + 9 –
  - 5 3 + 6 2 / * 3 5 * +
- *Algorithm to evaluate (use **stack**)*
  1. Push the operands in a stack
  2. When you encounter a operator
     - Pop 2 operands
     - Perform the operation on operands
     - Push the result in stack
  - Repeat from 1 until expression is parsed
  - The last item in stack is the answer
    - There would be only 1 item left when done correctly
- * prefix expr can also be evaluated in this way (*just in **reverse***)

Evaluate

- Prefix
  - - * 5 + - 4 2 2 / 6 3
  - - * + 3 5 7 + / 4 2 1
  - - + 10 * 2 3 + 4 / 5 5
- Postfix
  - 1 2 + 3 4 + * 5 6 - / 7 +
  - 8 2 / 3 4 + * 5 1 + 2 / -
  - 9 8 4 2 1 ^ * / - 3 +

Answers

- Prefix
  - - * 5 + - 4 2 2 / 6 3
    - Ans : **18**
  - - * + 3 5 7 + / 4 2 1
    - Ans : **53**
  - - + 10 * 2 3 + 4 / 5 5
    - Ans : **11**
- Postfix
  - 1 2 + 3 4 + * 5 6 - / 7 +
    - Ans : **-14**
  - 8 2 / 3 4 + * 5 1 + 2 / -
    - Ans : **25**
  - 9 8 4 2 1 ^ * / - 3 +
    - Ans : **11**

Practice: Programming

- Write a function that takes a string and checks if it is a palindrome
  - Returns true if palindrome is found else false
  - Hints: string indexing, loops, if , comparing characters
- Count the number of vowels, consonants in a sentence.
  - Vowels : a,e,i,o,u
  - Consonant : everything else other than vowels
  - Ignore : spaces (‘ ’,) comma(,) dash(-),semicolon(;),colon(:)
- Find the most frequent word in a sentence
  - If more than 1 word has same frequency return the lexicographically smaller one.

Review Evaluating Prefix/Postfix

- From last week
  - Operator
  - Operand
  - Stack
  - Queue
- Evaluating Prefix/Postfix
  - <3 questions each>

Precedence and associativity

- Precedence (highest priority first)
  - ()
  - ^
  - / *
  - - +
- Associativity
  - Right to left : ^
  - Left to right : + , - , * , /
- Notes
  - () not categorized as associative, they represent grouping mechanism

Precedence and associativity

- Example
  - 10 + 20 / 2 * 3 – 5
    - ➔ 10 +((20 / 2) * 3)-5 // added parenthesis for denoting precedence
    - ➔ 10 + (10 * 3) -5 // interior most precedence first
    - ➔10 + 30 - 5
    - ➔ **35**

Precedence and associativity (lets evaluate)

- 2 + 3 * 4 ^2
- 2 ^ 3 ^ 2
- 100 / 2 * 5 ^ 2 ^ 1

Precedence and associativity (answers)

- 2 + 3 * 4 ^2
  - ➔ 2 + 3 * 16
  - ➔ 2 + 48
  - ➔ **50**
- 2 ^ 3 ^ 2
  - ➔ 2 ^ 9 // ^ is evaluated from right to left , so 3 ^ 2 goes first
  - ➔ **512**
  - ~~Not 64~~
- 100 / 2 * 5 ^ 2 ^ 1
  - ➔ 100 / 2 * 25 // Again 2 ^ 1 goes first
  - ➔ 50 x 25
  - ➔**1250**

Convert to Prefix/Postfix (examples)

- B * C
  - Prefix : *** B C**
  - Postfix : **B C ***
- A + B * C
  - Prefix : +**A*BC**
  - Postfix : **ABC*+**
- (5 + 6) * 3
  - Prefix : *** + 5 6 3**
  - Postfix : **5 6 + 3 ***

Convert to Postfix

- How to convert!
  - Convert the highest precedence/priority first
  - (do paranthesize for mental ease)
- (a + b) / (c – d) + e
- x + y / (g * h – k)

Convert to Postfix

- How to convert!
  - Convert the highest priority first
  - (and paranthesize for mental ease)
- (a + b) / (c – d) + e
  - Prefix : **+ / + a b – c d e**
  - Postfix : **a b + c d - / e +**
- x + y / (g * h – k)
  - Prefix : **+ x / y - * g h k**
  - Postfix : **x y g h * k - / +**

Convert to Prefix and Postfix

- a + b - c * d + e ^ f
- (a + b) * c - (d - e) * (f + g)
- ((a + b) * (c + d) / (e - f)) + g
- a * (b + c) / (d - e)
- a - b / (c * d ^ e)
- (a + b) * (c + d) - e

Convert to Prefix and Postfix (answers)

Infix	Prefix	Postfix
a + b - c * d + e ^ f	+ - + a b * c d ^ e f	a b + c d * - e f ^ +
(a + b) * c - (d - e) * (f + g)	- * + a b c * - d e + f g	a b + c * d e - f g + * -
((a + b) * (c + d) / (e - f)) + g	+ / * + a b + c d - e f g	a b + c d + * e f - / g +
a * (b + c) / (d - e)	/ * a + b c - d e	a b c + * d e - /
a - b / (c * d ^ e)	- a / b * c ^ d e	a b c d e ^ * / -
(a + b) * (c + d) - e	- * + a b + c d e	a b + c d + * e -

Logical operators (for bits/bytes)

- Bitwise
  - NOT, AND, XOR , OR
- Shift
  - Linear : LSHIFT, RSHIFT,
  - Circular : RCIRC, LCIRC
- Precedence
  - NOT,
  - SHIFT, CIRC,
  - AND ,
  - XOR,
  - OR

Truth Tables

X	NOT X
0	1
1	0

X	Y	X AND Y	X OR Y	X XOR Y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Truth Tables

- Linear shift , add 0s as padding (either left or right)
  - Shifting by word size (or more) would result in all 0s
- Circular shift just rolls over the bits (as per direction)
  - Shifting by the number of word size , same bit sequence would be produced

X	(LSHIFT-2 X)	(RSHIFT-3 X)	(LCIRC-3 X)	(RCIRC-1 X)
01101	10100	00001	01011	10110
10	00	00	01	01
1110	1000	0001	0111	0111
1011011	1101100	0001011	1011101	1101101

Operations (evaluate)

- 0110 AND 1011
- 1100 OR 0101
- 1010 XOR 1111
- 1010 AND 1100 OR 0001
- 0011 OR 1010 AND 0110
- (1100 XOR 1010) AND 0110

Operations (evaluate)

- 0110 AND 1011
- 1100 OR 0101
- 1010 XOR 1111
- 1010 AND 1100 OR 0001
- 0011 OR 1010 AND 0110
- (1100 XOR 1010) AND 0110

Operations : Shift (evaluate)

1. (LSHIFT-1 10101) AND (RSHIFT-2 11010)
2. (RCIRC-2 01111) OR (LSHIFT-3 10010)
3. (RSHIFT-1 11100) XOR (LCIRC-2 01011)
4. (LSHIFT-2 00110) OR (RCIRC-1 10101) AND (RSHIFT-3 11101)
5. (LCIRC-3 11001) AND (RSHIFT-2 10110)
6. (RCIRC-4 10011) OR (LSHIFT-1 01101)
7. (RSHIFT-3 11110) AND (LCIRC-1 00111) OR (LSHIFT-2 01010)
8. (LSHIFT-2 01101) OR (RCIRC-3 10110) AND (RSHIFT-1 11011)

Answsers 1..3

1. (LSHIFT-1 10101) AND (RSHIFT-2 11010)
   - → 01010 AND 00110
   - → **00010**
2. (RCIRC-2 01111) OR (LSHIFT-3 10010)
   1. → 11011 OR 10000
   2. → **11011**
3. (RSHIFT-1 11100) XOR (LCIRC-2 01011)
   1. → 01110 XOR 01101
   2. → **00011**

Answers 4..6

4. (LSHIFT-2 00110) OR (RCIRC-1 10101) AND (RSHIFT-3 11101)
   - 11000 OR 11010 AND 00011 *//parenthesis evaluated first*
   - 11000 OR 00010 *// And has higher precedence so evaluated first*
   - **11010**
5. (LCIRC-3 11001) AND (RSHIFT-2 10110)
   - 01110 AND (RSHIFT-2 10110)
   - 01110 AND 00101
   - **00100**
6. (RCIRC-4 10011) OR (LSHIFT-1 01101)
   - 00111 OR 11010
   - **11111**

Answers 7..8

7. (RSHIFT-3 11110) AND (LCIRC-1 00111) OR (LSHIFT-2 01010)
   - 00011 AND 01110 OR (01000)
   - 00010 OR 01000
   - **01010**
8. (LSHIFT-2 01101) OR (RCIRC-3 10110) AND (RSHIFT-1 11011)
   - 11010 OR 11010 AND 01101
   - 11010 OR 01000
   - **11010**

What does this Program do ? Q1.

```
num = 72
for i = 2 to 6
    count = 0
    while num % i == 0
        num = num / i
        count = count + 1
    end while
    if count > 0 then
        output i, count
    end if
```

What does this Program do ? A1.

- i = 2
  - num = 72
  - 72 % 2 == 0 → yes
    - num = 36, count = 1
  - 36 % 2 == 0 → yes
    - num = 18, count = 2
  - 18 % 2 == 0 → yes
    - num = 9, count = 3
  - 9 % 2 != 0 → stop
  - output → 2 3
- I =3
  - output → 3 2
- I = 4 ,5, 6
  - num = 1 (not divisible by any)
  - no output

```
num = 72
for i = 2 to 6
    count = 0
    while num % i == 0
        num = num / i
        count = count + 1
    end while
    if count > 0 then
        output i, count
    end if
```

What does this Program do ? Q2.

```
sum = 0
for i = 1 to 5
    for j = 1 to i
        sum = sum + j
    next j
next i
output sum
```

What does this Program do ? A2.

- Nested loop sum : 35

i	j	*Added*	sum
1	1	1	1
2	1+2	3	4
3	1+2+3	6	10
4	1+2+3+4	10	20
5	1+2+3+4+5	15	**35**

```
sum = 0
for i = 1 to 5
    for j = 1 to i
        sum = sum + j
    next j
next i
output sum
```

What does this Program do ? Q3

```
x = 0
for i = 10 to 20
    if i % 3 == 0 and i % 4 != 0 then
        x = x + i
    end if
next i
output x
```

What does this Program do ? A3

- We want
  - Divisible by 3
  - NOT divisible by 4
- 15 + 18 = 33

i	%3	%4	Include?
12	Y	Y	No
15	Y	N	Yes
18	Y	N	Yes

```
x = 0
for i = 10 to 20
    if i % 3 == 0 and i % 4 != 0 then
        x = x + i
    end if
next i
output x
```

What does this Program do ? Q4

```
n = 2468
count = 0
while n > 0
    digit = n % 10
    if digit % 2 == 0 then
        count = count + 1
    end if
    n = n / 10
end while
output count
```

What does this Program do ? A4

- Output 4

Step	digit	even?	count
8	yes	1	1
6	yes	2	2
4	yes	3	3
2	yes	4	4

```
n = 2468
count = 0
while n > 0
    digit = n % 10
    if digit % 2 == 0 then
        count = count + 1
    end if
    n = n / 10
end while
output count
```

What does this Program do ? Q5

```
A = [3, 6, 9, 12, 15]
for i = 0 to 4
    if A[i] % 3 == 0 then
        A[i] = A[i] / 3
    end if
next i
output A
```

What does this Program do ? A5

- Divide every element by 3
- Output 1,2,3,4,5

Index	Original	New
0	3	1
1	6	2
2	9	3
3	12	4
4	15	5

```
A = [3, 6, 9, 12, 15]
for i = 0 to 4
    if A[i] % 3 == 0 then
        A[i] = A[i] / 3
    end if
next i
output A
```

What does this Program do ? Q6

```
A = [5, 1, 4, 2]
for i = 0 to 2
    if A[i] > A[i+1] then
        temp = A[i]
        A[i] = A[i+1]
        A[i+1] = temp
    end if
next i
output A
```

What does this Program do ? A6

- Single pass swap
  - Output : 1,4,2,5
  - **Note : not sorted**
- i =0
  - 5 > 1 → swap → [1, 5, 4, 2]
- i =1
  - 5 > 4 → swap → [1, 4, 5, 2]
- i = 2
  - 5 > 2 → swap → [1, 4, 2, 5]

```
A = [5, 1, 4, 2]
for i = 0 to 2
    if A[i] > A[i+1] then
        temp = A[i]
        A[i] = A[i+1]
        A[i+1] = temp
    end if
next i
output A
```

What does this Program do ? Q7

```
n = 5
result = 1
for i = 1 to n
    result = result * i
    if result > 50 then
        output i
        break
    end if
next i
```

What does this Program do ? A7

- Output 5

Step	i	result	action
1	1	1	result < 50 continue
2	2	2	result < 50 continue
3	3	6	result < 50 continue
4	4	24	result < 50 continue
5	5	120	**result > 50 break**

```
n = 5
result = 1
for i = 1 to n
    result = result * i
    if result > 50 then
        output i
        break
    end if
next i
```

Boolean Logic/Algebra

- The inputs are : 0,1 {only}
- The outputs are : 0,1 {only}
- Operators
  - ()
  - NOT → $\bar{A}$
  - AND→ $A \,.\, B$
  - XOR →$A \oplus B$
    - XNOR → $\overline{A \oplus B}$
  - OR →$A + B$

Truth Tables (same as bitwise operations → easy)

X	$\overline{X}$
0	1
1	0

X	Y	$X.Y$	$X+Y$	$X \oplus$ Y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Some expressions

- Evaluate for A = 1 , B = 0 , C =0
  - $A.(B + C)$
  - $\bar{A}.(B + C)$
  - $A.(\bar{B} + C)$
  - $(A + B)(A + C)$
  - $A + \bar{A}.\mathrm{B}$
  - $A + AB$

Rules/Discovery

- Identity
  - A + 0 = A
  - A . 1 = A
  - A + 1 = 1
  - A . 0 = 0
- Idempotent
  - $A + A$ = A
  - A . A = A
- Complement
  - $A + \bar{A}$ = 1
  - A . $\overline{A}$ = 0

Some more rules

- Double Negation
  - $\bar{\bar{A}} = A$
- Absorption law
  - $A + A.B = A$
  - $A.(A + B) = A$
- **Distribution law**
  (symmetrical to both . and +)
  - $A.(B + C) = (A.B) + (B.C)$
  - $A + (B.C) = (A + B).(A + C)$

Do it yourself

- Hint : These may be actual theorems
  - $\overline{A.B} = ?$
  - $\overline{A+B} = ?$
  - $(A+B).(\bar{A}+C) = ?$
  - $A(\bar{A}+B) = ?$
  - $A+\bar{A}B = ?$

Solved (and named)

- Hint : These may be actual theorems
  - $\overline{A.B} = \bar{A} + \bar{B}$
    - DeMorgan's law
  - $\overline{A + B} = \bar{A}.\bar{B}$
    - DeMorgan's law (again)
  - $(A + B).(\bar{A} + C) = AC + \bar{A}\mathrm{B}$
    - Consensus theorem
  - $A(\bar{A} + B) = A.B$
    - Redundancy law
    - Also called, Second Absorption law
  - $A + \bar{A}B = A + B$
    - Redundancy law (again)
    - Also called , Third distributive law

Lets Simplify 1..4

- $A.(A+B)$
- $A+(A.B)$
- $\bar{A}+(A.B)$
- $(A+B).(A+\bar{B})$

Lets Simplify 1..4 : answers

- $A.(A+B)$
  - By Absorption law = A, OR
    - A.A + A.B
    - A + AB //(A.A = A), Idempotent Law
    - A (1+ B) // factor A out
    - A // By null law 1+B = 1 and A.1 = A
- $A+(A.B)$
  - A
- $\bar{A}+(A.B)$
  - $(\bar{A}+A).(\bar{A}+B)$ //Distribution law
  - $1.(\bar{A}+B)$ // By Complement law $(\bar{A}+A)=1$
  - $\bar{A}+$ B
- $(A+B).(A+\bar{B})$
  - $(A.A)+(B.\bar{B})$ //Distribution law
  - A+0 //Idempotent law + Complement law
  - A

Lets Simplify 5..11

- $(A.B) + (A.\bar{B}) + (\bar{A}.C)$
  - $\boldsymbol{A + C}$
- $(A + B).(A + \bar{B}).(\bar{A} + B)$
  - $\boldsymbol{A.B}$
- $\overline{A + B} + A.\bar{B}$
  - $\boldsymbol{\bar{B}}$
- $(A + B + C).(\bar{A} + B).(A + \bar{C})$
  - $\boldsymbol{B.(A + \bar{C})}$
- $(A.\bar{B} + B.C).(\bar{A} + \bar{C})$
  - $\boldsymbol{A.\overline{BC} + \bar{A}.B.C}$
- $\overline{(A.B)}.(\bar{A} + C) + (A.\bar{C})$
  - $\boldsymbol{\bar{A} + \bar{B} + \bar{C}}$

Lets Simplify 5..11 : answers

- These are hard , think of them as puzzles,
  - $(A.B) + (A.\bar{B}) + (\bar{A}.C)$

  - $(A + B).(A + \bar{B}).(\bar{A} + B)$
  - $\overline{A + B} + A.\bar{B}$
  - $(A + B + C).(\bar{A} + B).(A + \bar{C})$
  - $(A.\bar{B} + B.C).(\bar{A} + \bar{C})$
  - $\overline{(A.B)}.(\bar{A} + C) + (A.\bar{C})$

Truth Tables/Expressions

- Create truth table for each and validate/discover simplifications
  1. $A.(A+B)$ = A
  2. $A+(A.B)$ = A
  3. $\bar{A}+(A.B)$ = $\bar{A}+$ B
  4. $(A+B).(A+\bar{B})$ = A
  5. $(A+B).(A+\bar{B}).(\bar{A}+B) = ?$
  6. $\overline{A+B}+A.\bar{B} = ?$
  7. $(A+B+C).(\bar{A}+B).(A+\bar{C}) = ?$
  8. $(A.\bar{B}+B.C).(\bar{A}+\bar{C}) = ?$

Expressions: Soln

- Solutions to Boolean Simplifcations ....
  1. $A.(A+B)$ = **A** // Use Absorption law
  2. $A+(A.B)$ = **A** // Use Absorption law, *other one*
  3. $\bar{A}+(A.B)$ = $\bar{A}+B$
     - ➔ $(\bar{A}+A).(\bar{A}+B)$ //Distribution Law
     - ➔ (1). $(\bar{A}+B)$ // Idempotent Law
     - ➔$(\bar{A}+B)$
  4. $(A+B).(A+\bar{B})$ = A
  5. $(A+B).(A+\bar{B}).(\bar{A}+B)=AB$
  6. $\overline{A+B}+A.\bar{B}=\bar{B}$
  7. $(A+B+C).(\bar{A}+B).(A+\bar{C})=B.(A+\bar{C})$
  8. $(A.\bar{B}+B.C).(\bar{A}+\bar{C})=A\overline{BC}+\bar{A}BC$

Expressions: Soln 1..4

- Solutions to Boolean Simplifcations ....
  1. $A.(A+B)$ = **A** // Use Absorption law
  2. $A+(A.B)$ = **A** // Use Absorption law, *other one*
  3. $\bar{A}+(A.B)$ = $\bar{A}+B$
     - ➔ $(\bar{A}+A).(\bar{A}+B)$ //Distribution Law
     - ➔ (1). $(\bar{A}+B)$ // Idempotent Law
     - ➔$(\bar{A}+B)$
  4. $(A+B).(A+\bar{B})$ = A
     - ➔$A+(B.\bar{B})$ //Distribution Law *in reverse*
     - ➔$A+0$ //Complement Law
     - ➔A

Expressions: Soln 5,6

5. $(A+B).(A+\bar{B}).(\bar{A}+B) = AB$
   - ➔ $(A+B.\bar{B}).(\bar{A}+B)$ //See 4
   - ➔ $(A\ ).(\bar{A}+B)$ //Complement Law
   - ➔ $A.\bar{A}\ + A.B$ //Distribution Law
   - ➔ $A.B$ //Complement Law
6. $\overline{A+B} + A.\bar{B} = \bar{B}$
   5. $\bar{A}.\bar{B} + A.\bar{B}$ //De-Morgan's Law
   6. $(\bar{A}+A).\bar{B}$ //Factor the like terms
   7. $\bar{B}$ //Identity Law
7. $(A+B+C).(\bar{A}+B).(A+\bar{C}) = B.(A+\bar{C})$
8. $(A.\bar{B}+B.C).(\bar{A}+\bar{C}) = A\overline{BC} + \bar{A}BC$

Expressions: Soln 5,6

7. $(A + B + C).(\bar{A} + B).(A + \bar{C}) = B.(A + \bar{C})$
   - ➔ $(A.(\bar{A} + B) + B.(\bar{A} + B) + C.(\bar{A} + B)) . (A + \bar{C})$
   - ➔ $(A.B + B + C.\bar{A} + C.B) . (A + \bar{C})$ //Absorption and expansion
   - ➔ $(B + C.\bar{A}) . (A + \bar{C})$ //Absorption
   - ➔ $A.B + B.\bar{C} + C.\bar{A}.A + CA\bar{C}$ //Complement Law and combine like terms
   - ➔ $B.(A + \bar{C})$
8. $(A.\bar{B} + B.C).(\bar{A} + \bar{C}) = A\overline{BC} + \bar{A}BC$
   - ➔ $A.\bar{B}.(\bar{A} + \bar{C}) + B.C.(\bar{A} + \bar{C})$ //Distribute
   - ➔ $A\overline{BA} + A\overline{BC} + B.C.(\bar{A} + \bar{C})$
   - ➔ $A\overline{BC} + B.C.(\bar{A} + \bar{C})$ //Complement Law
   - ➔ $A\overline{BC} + B.C.\bar{A} + B.C.\bar{C}$ //Distribute
   - ➔ $A\overline{BC} + B.C.\bar{A}$ //Complement Law and rearrange

Programming Practice

- Specific to Test2 ,*(skip if you are already done with test 2)*
  - https://www.hackerrank.com/acsltest2practise
- Specific to Test 3
  - https://www.hackerrank.com/acsltest3practise
  - Knowledge needed
    - New: Stacks and Queues
    - Existing: if else, loops, arrays, functions, …

Data Structures : Queue

- FIFO : First In First Out
  - Push
    - Action: Push an item into queue
    - Can also be called (enqueue , push_back, ...)
  - Pop
    - Action :Remove the first item in queue
    - Can also be called (dequeue , pop_front,...)
  - Top
    - Action : Peek (do not remove) the first item in queue
    - Can also be called (front, ...)
  - Simulation
    - ems_winter_2025_acsl101\source\queue_stack

Data Structures : Stack

- LIFO : Last In First Out
  - Push
    - Action: Push an item into stack
    - Can also be called (insert,push_back, ...)
  - Pop
    - Action :Remove the first item in queue
    - Can also be called (pop_front,...)
  - Top
    - Action : Peek (do not remove) the first item in queue
    - Can also be called (back, ...)
  - Simulation
    - ems_winter_2025_acsl101\source\queue_stack

What does this program do ? Q1

```
A = [1, 2, 3, 4, 5]

for i = 0 to 4
    A[i] = A[i] + i
next i

for i = 4 down to 0
    output A[i]
next i
```

What does this program do ? A1

9 7 5 3 1

i	A[i] : Before	A[i] : After
0	1	1
1	2	3
2	3	5
3	4	7
4	5	9

```
A = [1, 2, 3, 4, 5]

for i = 0 to 4
    A[i] = A[i] + i
next i

for i = 4 down to 0
    output A[i]
next i
```

What does this program do ? Q2

```
A = [2, 4, 6, 8]

for i = 0 to 2
    for j = i+1 to 3
        if A[i] > A[j] then
            swap A[i], A[j]
        end if
    next j
next i

output A
```

What does this program do ? A2

2 4 6 8

- This program tried to sort
- Already sorted →Nothing happens

```
A = [2, 4, 6, 8]

for i = 0 to 2
    for j = i+1 to 3
        if A[i] > A[j] then
            swap A[i], A[j]
        end if
    next j
next i

output A
```

What does this program do ? Q3

```
sum = 0

for i = 1 to 4
    for j = 1 to i
        if (i + j) % 2 == 0 then
            sum = sum + j
        end if
    next j
next i

output sum
```

What does this program do ? A3

- **13**
- Add j only when (i+j) is even
  - i =1
    - j=1 → 2 (even) → +1 → sum=1
  - i=2
    - j=1 → 3
    - j=2 → 4(even) → +2 → sum=3
  - i=3
    - j=1 → 4 (even) → +1 → sum=4
    - j=2 → 5 (odd)
    - j=3 → 6 (even) → +3 → sum=7
  - i=4
    - j=1 → 5 (odd)
    - j=2 → 6 (even) → +2 → sum=9
    - j=3 → 7 (odd)
    - j=4 → 8 (even) → +4 → sum=13

```
sum = 0

for i = 1 to 4
    for j = 1 to i
        if (i + j) % 2 == 0 then
            sum = sum + j
        end if
    next j
next i

output sum
```

What does this program do ? Q4

```
A = [1, 3, 5, 7]

count = 0

for i = 0 to 3
    for j = 0 to 3
        if A[i] < A[j] then
            count = count + 1
        end if
    next j
next i

output count
```

What does this program do ? A4

- 6
- Count how many times A[i] < A[j]
  - 1 < 3,5,7 → 3
  - 3 < 5,7 → 2
  - 5 < 7 → 1
  - 7 < none → 0
- Total = 3 + 2 + 1 = 6

```
A = [1, 3, 5, 7]

count = 0

for i = 0 to 3
    for j = 0 to 3
        if A[i] < A[j] then
            count = count + 1
        end if
    next j
next i

output count
```

What does this program do ? Q5

```
stack S

for i = 1 to 4
    push i
next i

for i = 1 to 2
    pop
next i

push 5

while S not empty
    output top(S)
    pop
end while
```

What does this program do ? Q5

- 5 2 1
  - Push 1,2,3,4
  - Pop twice
    - 4 → 1,2,3
    - 3 → 1,2
  - Push 5
    - 1,2,5
  - Pop all
    - 5,2,1

```
stack S

for i = 1 to 4
    push i
next i

for i = 1 to 2
    pop
next i

push 5

while S not empty
    output top(S)
    pop
end while
```

What does this program do ? Q6

```
A = [3, 1, 4, 2]

for i = 0 to 3
    for j = 0 to 2
        if A[j] > A[j+1] then
            swap A[j], A[j+1]
        end if
    next j
next i

output A
```

What does this program do ? A6

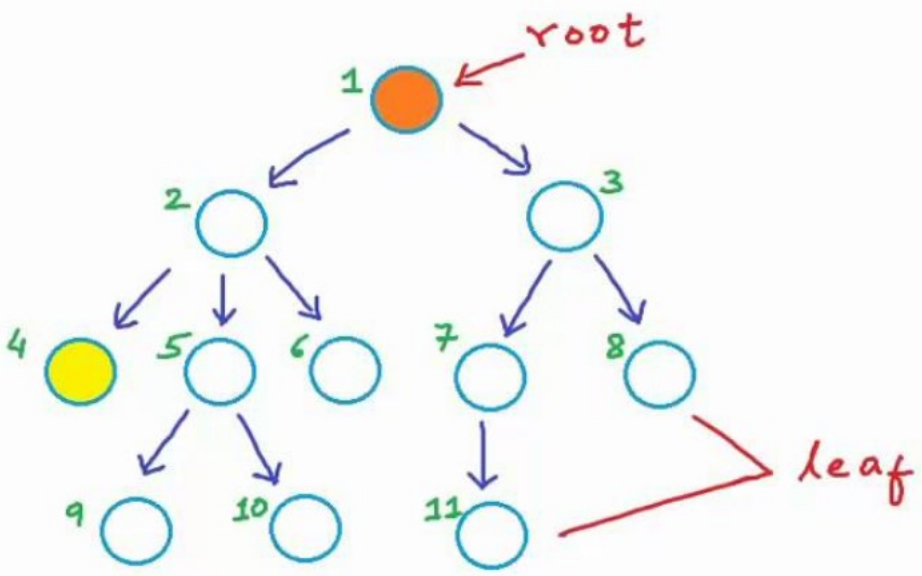
- 1 2 3 4
  - Bubble sort
    - Pass 1 → 1,3,2,4
    - Pass 2 → 1,2,3,4
    - Pass 3 → (No Change)
    - Pass 4 → (No Change)

```
A = [3, 1, 4, 2]

for i = 0 to 3
    for j = 0 to 2
        if A[j] > A[j+1] then
            swap A[j], A[j+1]
        end if
    next j
next i

output A
```

Data structure : Trees

- Not the green/woody kind
  - But they actually look like upside down trees
- Trees are data structures that represents data in parent/child relationships
  - Start with a **root** node
  - (May) point to other nodes
    - This is **parent** pointing to **child** (generally) or vice versa
  - The nodes with no children are **leaf** nodes

Tree: Visuals



root
children
Parent
Sibling → have same parent
leaf → has no child

Trees : types

- Based on child count
  - Binary trees (most popular)
    - Each node max 2 children (i.e 0,1 or 2)
  - Ternary trees
    - Max 3 children
  - N-ary
    - Max N children

Trees: Types

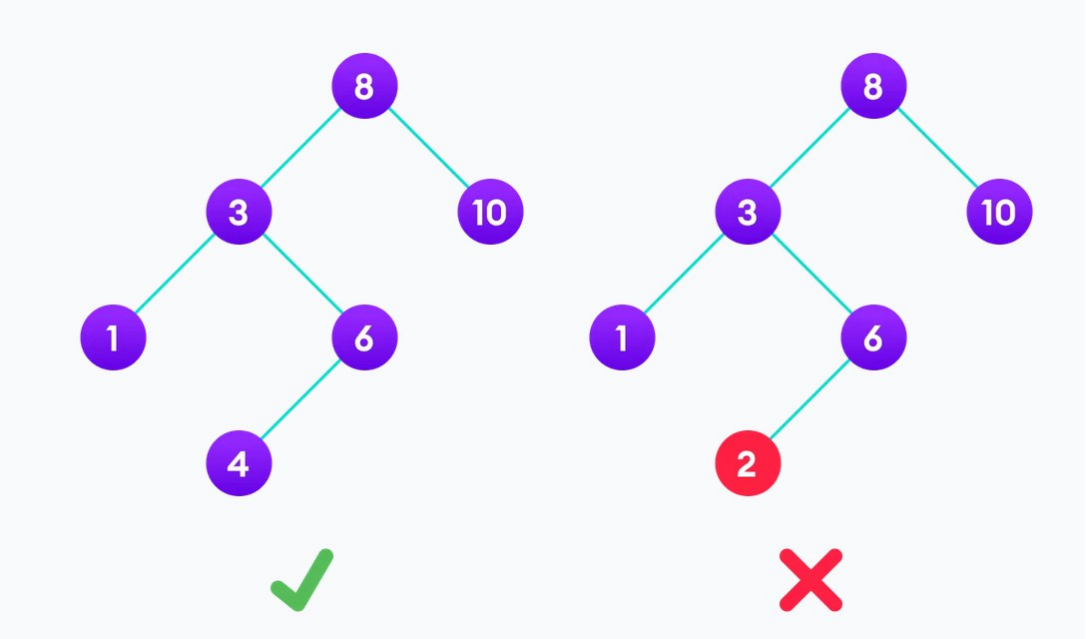
- Based on data organization
  - Binary Tree
    - No organization requirement
  - Binary Search trees ,
    - Left child is smaller (or equal to) the root
    - Right child is bigger than the root
  - **AVL tree** (Adelson-Velsky and Landis)
    - Binary search tree with limits on height
    - No one leaf can be more than a 1 distance logner than any other leaf node
  - Red black tree
    - A representation of AVL tree that color codes the nodes (red/black) to organize info.
  - …

Binary Tree : Key terms

- Root : there is only 1 root(0 for empty tree)
- Parent
  - All other nodes have one parent (except the root node)
- Children
  - In Binary Tree : there can be at most 2 children
  - The 2 children are also referred as siblings
- Leaf nodes : nodes with no children are leaf nodes
- Height : the distance from the root to furthest leaf node is height of the tree

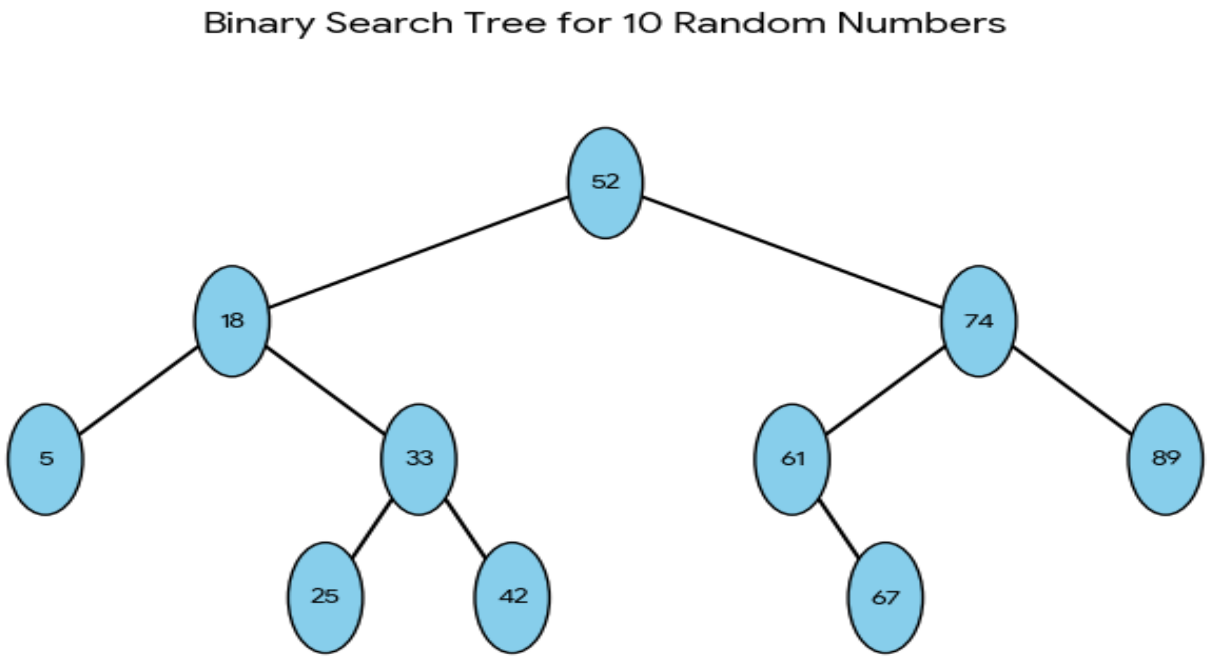
Binary Search tree : Examples

- Left child must be smaller than or equal to the parent
- Right child must be larger than the parent



Creation of a BST

- **52, 18, 74, 5, 33, 61, 89, 25, 42, 67**

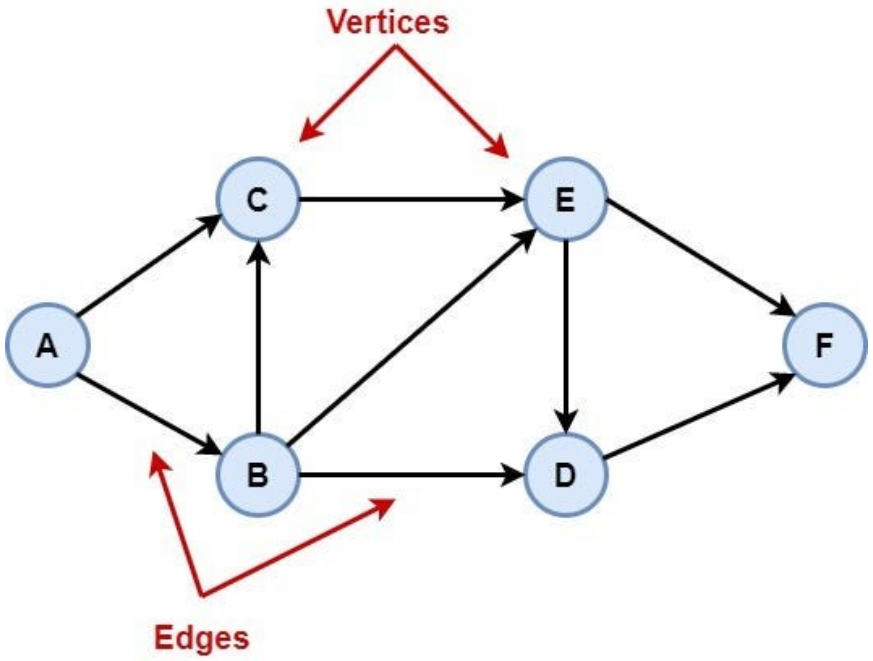


Binary Search Tree : exercise

- Create a tree for
  - S U P E R N A T U R A L
  - 16 93 81 37 98 84
  - 0 9 2 6 2 3 5 1 4 2 9 7 6 8 9 2 7
  - C O N I O S I S
  - A L P H A B E T I C A L
  - M A T H E M A T I C S

Data Structure : Graphs

- Graphs are data structure that can represent complex relationships between items. They are made of
  - Vertices(also called nodes)
  - Edges

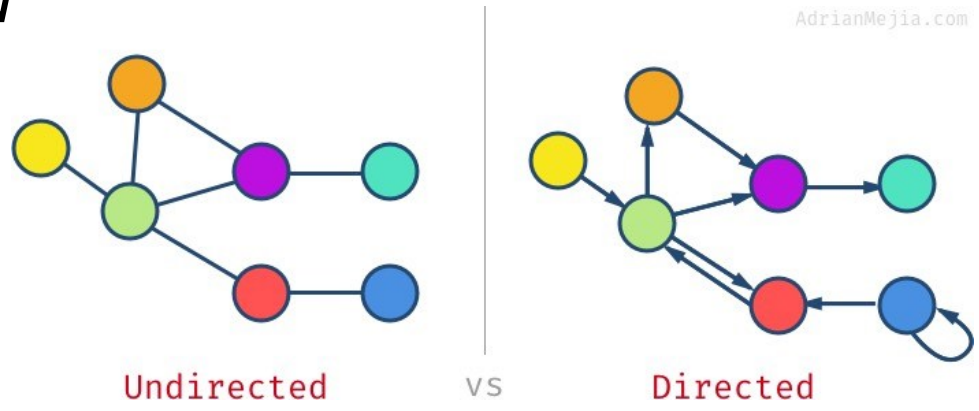


Graphs : uses

- Major part of day to day life
  - Maps
    - Shortest/fastest routes
  - Internet/Computer networks
    - Communication over internet
  - Commerce
    - Stocks
    - goods
  - Social Networks
    - Friends
    - Ads
  - Circuits
    - Computer
  - Schedules
    - dependency

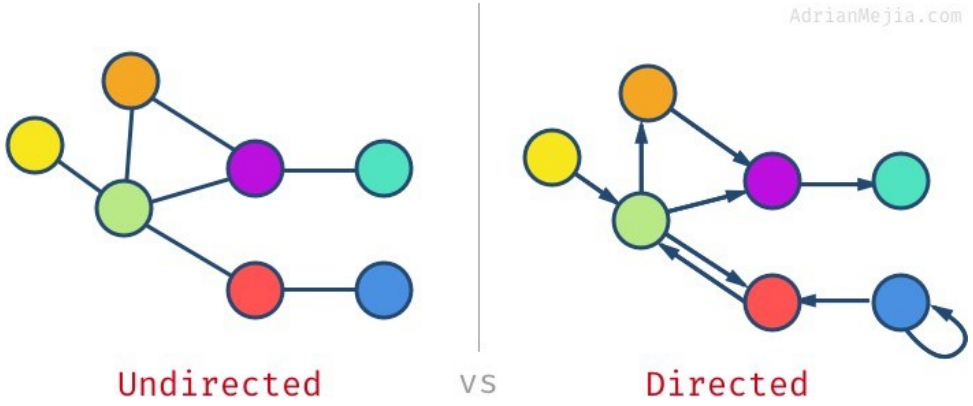
Graphs : by Edge types

- **Directed** : think one way
  - The edges have a **source** where they begin from
  - And a **destination** where they end.
  - ***Think of this as one way connection***
- *Undirected*



Graphs : by Edge types

- Directed
- **Undirected**
  - The edges do not have direction
  - The edge connects vertices in both direction
  - ***Think of this as two way connection***



Graph representation(s)

Assuming **graph with ‘V’ vertices and ‘E’ edges**

- Adjacency matrix *(double dimension array)*
  - VxV array, entry in row ‘i’ column ‘j’ denotes connectivity between node i : j
  - *Very useful and efficient representation*
- array of edges
  - Array of size ‘E’ containing source and destination of edges
  - *Simple but slower representation*
- Array of adjacency lists
  - Vertex indexed array of edges.
  - *Flexible and quite efficient* (the goat)
- …(and there are more representations)

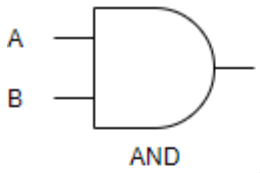
In class exercises

- …

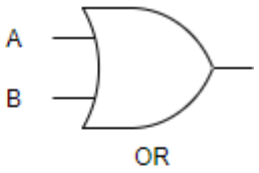
Logic gates

- The Cictuits in electronics (Pcs, gaming consoles, phones) are made of basic logic units, called **logic gates**
  - Basic : **AND, OR, NOT , XOR**
  - Negated : **NAND, NOR , XNOR**
    - Similar to AND , OR, XOR respectively but with extra NOT at the end
  - *There are more custom gates but these are basic ones*
- These are visual representation of the Boolean algebra
- All the complex calculations you can think of are fundamentally run by these gates

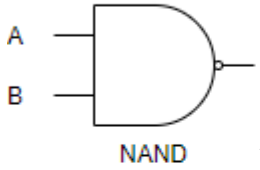
Logic gates



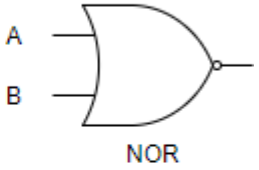
$A\,.\,B$



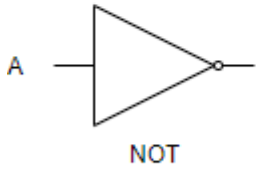
$A + B$



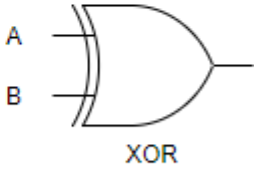
$\overline{A.B}$



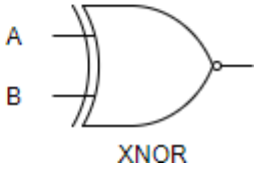
$\overline{A + B}$



$\bar{A}$



$A \oplus B$



$\overline{A \oplus B}$

In class exercises

- Boolean Expression…